Лабораторная работа № 7.
Работа с БД с использованием JDBC

Цель работы

Получить общее представление о прикладном программном интерфейсе JDBC для работы с БД. Освоить процедуру установки соединения с БД.

1. Теоретические сведения

Описание установки и настройки необходимого программного обеспечения приведено в Приложении 1.

1.1 Общие сведения о JDBC

JDBC (Java Database Connectivity) – **прикладной программный интерфейс** (application programming interface - API) для доступа к реляционным базам данных. JDBC позволяет выполнять соединение с базой данных, посылать запросы на получение и изменение данных, а также выполнять манипуляции с полученными данными. С технической точки зрения API JDBC представляет набор классов в библиотеке java.sql. Для работы с конкретной СУБД через JDBC необходим соответствующий драйвер.

JDBC - это «родной» для Java интерфейс к базовым абстракциям и концепциям языка SQL Использование JDBC позволяет разрабатывать переносимые приложения на Java, не зависящие от конкретной СУБД.

Рассмотрим использование JDBC на примере работы с СУБД PostgreSQL. Рекомендуется устанавливать версии PostgreSQL 15 или выше.

Примечание. Для других СУБД есть особенности, которые описаны в приложении 3.

1.2 Подготовка для работы с БД

Для работы с БД необходимо использовать один из двух вариантов:

1) на компьютер установить соответствующую СУБД (в этом случае сервер БД будет на локальном компьютере)

2) если сервер БД находится на сетевом компьютере, то на локальном компьютере надо установить клиентскую часть СУБД.

Для локальной СУБД на компьютере будет создан сервис (служба), который автоматически будет запускать сервер БД при старте операционной системы.

Для Maven проекта с API JavaFX драйвер подключается через зависимости в файле pom.xml (см. Приложение 2)

1.3 Соединение с БД

Для работы с БД приложению необходимо установить соединение с сервером БД. В ходе сессии соединения приложения посылает на сервер SQL-запросы и получает результаты (наборы данных). Для установления соединением и работы с БД используются классы: `DriverManager`,`Connection`.

Класс `DriverManager` по заданным параметрам ищет драйвер JDBC, используемый для соединения с БД.

Класс `Connection` используется для соединения с БД, а его методы обеспечивают выполнение запросов к БД .

Стандартный способ получения соединения - это вызов статического метода `getConnection` класса `DriverManager`:

```
DriverManager.getConnection(url, user, pwd)
```

где `user` - логин, `pwd` - пароль, `url` - строка, по которой JDBC определяет параметры БД, с которой устанавливается соединение.
Строка `url` имеет следующий формат:

```
<протокол:><подпротокол:> [<имя хоста:порт>]<имя БД>[?<схема>]
```

где
`протокол` – всегда `jdbc`;
`подпротокол` – тип БД. Для работы с БД Postgres - это `postgresql`;
имя хоста – IP-адрес или dns-имя сервера БД;
порт - для Postgres - 5432

Пример строки соединения (имя БД testdb, а имя схемы prod)
а) локальный доступ
`jdbc:postgresql://localhost:5432/testdb?currentSchema=prod`
б)сетевой доступ
`jdbc:postgresql://asoiumn1.asoiu:5432/testdb?currentSchema=prod`

1.4 Выполнение запросов

Установленное соединение используется для выполнения SQL-запросов к БД.

В JDBC есть классы, объекты которых используются для отправки SQL-запросов в БД:

1) Объект класса `Statement` - создается методом `createStatement` класса `Connection` и используется для выполнения простых SQL-запросов.

2) Объект класса `PreparedStatement` - создается методом `prepareStatement` класса `Connection`. Объект данного класса используется в SQL-запросах с параметрами. Класс `PreparedStatement` содержит группу методов, устанавливающих значения входных параметров запроса. Объект класса `PreparedStatement` потенциально может быть

более эффективным, чем объект класса `Statement`, так как он предварительно компилируется и сохраняется для будущего использования.

1.5 Последовательность действий для работы с БД

Для того чтобы подключиться к СУБД и выполнить запрос, необходимо написать код для выполнения ***следующих действий***:

1) Подключение необходимых библиотек (пакетов) директивой `import`
2) Установка соединения с БД
3) Создание объекта запроса
4) Выполнение запроса и получение результирующего набора (при необходимости). В случае получения набора данных выполняется шаг 5
5) Обработка полученного набора
6) Закрытие набора и объекта запроса
7) Закрытие соединения

Рассмотрим выполнение указанных действий более подробно.

Шаг 1. Подключение необходимых библиотек

Необходимо подключить стандартный пакет JDBC, используя директиву:

```
import java.sql.*;
```

Для подключения драйвера СУБД PostgreSQL необходимо указать положение драйвера JDBC в переменной `classpath`.

В проекте Maven драйвер добавляется автоматически, путем указания зависимости в файле pom.xml.

```
<dependency>
        <groupId>org.postgresql</groupId>
        <artifactId>postgresql</artifactId>
        <version>42.7.5</version>
</dependency>
```

Шаг 2. Установка (открытие) соединения

Необходимо создать соединение вызовом метода `getConnection` класса `DriverManager`

```
Connection conn = DriverManager.getConnection(
            строка_соединения, пользователь, пароль)
```

В этом методе пользователь (log) и пароль (pwd) задаются в окне диалога регистрации приложения БД, а строка соединения (url) может быть задана в приложении или считана из файла.

Шаг 3. Создание объекта запроса

Для создания объекта запроса класса `Statement` используется метод `createStatement` класса соединения

```
Statement stmt = con.createStatement();
```

Шаг 4. Выполнение запроса

Для выполнения запроса к БД используется метод `executeQuery` класса `Statement`. Полученный результат возвращается в виде объекта класса `ResultSet`.

Пример. Выбор данных из таблицы БД `Sprod`.

```
ResultSet rset = stmt.executeQuery ("SELECT * FROM Sprod");
```

Шаг 5. Обработка набора данных

Объект `ResultSet` поддерживает курсор - указатель на текущую строку набора данных. При получении набора данных курсор позиционируется перед первой строкой набора. Для изменения положения курсора используется метод набора данных `next()`, который перемещает курсор на следующую запись. Если записи исчерпаны, то метод возвращает значение `false`. Просмотр строк полученных данных выполняется в цикле до тех пор, пока не будет достигнут конец набора данных. Для получения значения колонки текущей строки используется метод `getXXX()` класса `ResultSet`, где XXX – предопределенный тип Java. Нумерация колонок начинается с 1. Допускается обращение к колонке по имени.

Пример. Вывод в консоль из каждой строки полученного набора данных первой колонки символьного типа.

```
while (rset.next())
    System.out.println(rset.getString(1));
```

Шаг 6. Закрытие набора данных и оператора запроса

После обработки данных необходимо явно закрывать экземпляры типов `Statement и ResultSet` вызовом метода `close()`.

Более правильно проверять существование объекта перед вызовом оператора закрытия для исключения ситуации получения ошибки `NullPointerException`

```
if (rset != null) {rset.close();}
if (stmt != null) {stmt.close();}
```

Шаг 7. Закрытие соединение

После завершения работы необходимо закрыть соединение

```
conn.close()
```

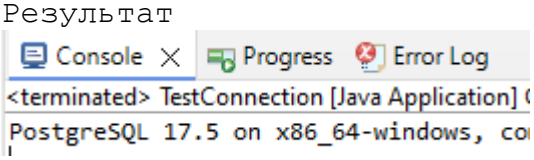
В программном приложении соединение обычно закрывается при завершении его работы

Пример. Соединение с СУБД и выполнение запроса на получение номера версии PostgreSQL

```
import java.sql.*;
public class TestConnection {
```

```
    public static void main(String[] args) {
      Connection con = null;
      Statement stmt = null;
      ResultSet res = null;
      String url =
         "jdbc:postgresql://localhost:5432/postgres";
      String user = "postgres";
      String password = "postgres";
    try {
     con = DriverManager.getConnection(url,user,password);
      stmt = con.createStatement();
      res = stmt.executeQuery("SELECT VERSION()");
      if (res.next()) {
           System.out.println(res.getString(1));
      }
    } catch (SQLException ex) {
        System.out.println(
         "Соединение не установлено\n"+ex.getMessage());
    } finally {
          if (res != null) {res.close();}
          if (stmt != null) {stmt.close();}
          if (con != null) {con.close();}
    }
  }
```

Результат



1.6 Создание простого приложения БД. Использование формы соединения

Обычно для соединения с БД используется диалоговое окно регистрации в приложении БД, в котором вводится имя пользователя и пароль.

Рассмотрение создания простого в приложения, в котором для соединения с БД используется окно диалога приведено в приложении 2.

Соединение с другими СУБД описано в приложении 3.

2. Задание

Изучить предлагаемый теоретический материал.

Создать приложение взаимодействия с БД, включающее окно авторизации и основное окно.

Протестировать работу приложения:

- выполнить из среды разработки;
- выполнить с использованием исполняемого jar-файла.

3. Содержание отчета

1) Краткое описание предметной области
2) Схема логической модели базы данных

3) Текст программы
4) Скриншоты окна авторизации и основного окна программы

4. Литература

1. Прохоренок Н.А. JavaFX. – СПб: ПХВ-Петербург, 2020
2. Учебное пособие по JavaFX .- https://www.geeksforgeeks.org/javafx-tutorial/
3. Информационные материалы. Доступ к базе данных JDBC. - https://docs.oracle.com/javase/tutorial/jdbc/.